Paper VIII
First Lean Verification Block

Packet Operator Algorithms, Exact Compilation,
and GPU Realization

Komorebi P. September 6, 2026

Scope and result

This supplement accompanies the strengthened manuscript dated August 14, 2026. It does not replace or silently revise that manuscript. The archived PDF, TeX source, bibliography, and original source package are preserved unchanged.

Ten named manuscript results are now covered by checked Lean statements. Five new modules contain 41 theorem declarations. An unchanged Paper VII module supplies 19 reused declarations and was recompiled. All 60 theorem audits and one separate coordinate-projector construction audit passed in the fresh final run.

Source item	Checked result	Module
2.3	Exact evaluation and matrix batches	`PacketIR`
2.5	Composition and support inclusion	`PacketIR`
2.6	Adjoint and actual tensor blocks	`PacketIR`
5.1	BDLR product and actual rank bound	`LowRankAlgebra`
5.3	Woodbury inverse and solve	`LowRankAlgebra`
5.4	Inverse operator-norm bound	`LowRankAlgebra`
6.1	Exact active-walk expansion	`TransferSupport`
6.2	Polynomial support	`TransferSupport`
6.3	Nilpotence from bounded paths	`TransferSupport`
8.1	Vector and operator error bounds	`PacketPerturbation`

Inventory boundary. The source register contains 56 theorem, proposition, and corollary records with manuscript proof environments. Forty-six remain not fully formalized in this release. Four classical inputs are counted separately. The six new boundary lemmas do not increase the number of completed manuscript items.

The original CPU numerical suite also passed all 37,392 checks in nine groups. These include consistency checks on historical benchmark records. No new GPU timing measurements were made, and those timings were not independently reproduced.

1. Exact packet semantics

Source items 2.3, 2.5, and 2.6

Let R be a ring, I a finite index type, and $(P_a)_{a\in I}$ a family satisfying

$$P_bP_a = \begin{cases} P_a & b = a, \\ 0 & b \neq a, \end{cases} \qquad \sum_a P_a = 1.$$

Write $A_{ba} = P_bAP_a$ and let $E_A = \{(b, a) : A_{ba} \neq 0\}$. These are exact algebraic zeros, not values discarded by a floating-point threshold.

Reconstruction and action. The checked finite sum is

$$\boxed{A = \sum_{(b,a)\in E_A} A_{ba}.}$$

The proof inserts the complete projector sum on both sides and removes only zero summands. Applying this identity to an arbitrary module gives exact action. A separate finite-matrix theorem establishes

$$AZ = \sum_{(b,a)\in E_A} A_{ba}Z$$

for every rectangular batch of right-hand sides Z.

Composition. Inserting the projector identity between factors yields

$$\boxed{(AB)_{ba} = \sum_c A_{bc}B_{ca}.}$$

If the left side is nonzero, at least one intermediate c has both $A_{bc} \neq 0$ and $B_{ca} \neq 0$. Consequently $\operatorname{supp}(AB) \subseteq \operatorname{supp}(A) \circ \operatorname{supp}(B)$. No equality of supports is asserted.

Adjoint and tensor products. With an involution and self-adjoint projectors, the formalization proves

$$(A_{ba})^* = (A^*)_{ab}.$$

For actual finite matrices and actual Kronecker products, it proves

$$(P_b \otimes Q_d)(A \otimes B)(P_a \otimes Q_c) = A_{ba} \otimes B_{dc}.$$

The self-adjointness hypothesis is explicit in the adjoint theorem. The tensor theorem is not a statement about an uninterpreted tensor symbol.

Concrete coordinates. Given any tag map from finite scalar coordinates to packet addresses, `coordinatePackets` constructs diagonal zero-one matrices. Their orthogonality and completeness are proved. The construction allows unequal packet sizes and unused packet addresses. Its action is checked entrywise: it retains exactly the coordinates carrying the selected tag.

The underlying ring-valued projector calculus is imported from the unchanged, freshly recompiled Paper VII foundation. Reuse is recorded separately from the new Paper VIII declarations.

2. Support propagation and nilpotence

Source items 6.1, 6.2, and 6.3

A path of length n in the Lean predicate `TransferPath` is a directed walk: vertices may repeat. This is the notion required by matrix powers. At length zero the endpoints coincide. At length $n+1$, a nonzero last block is appended to a length-n walk.

Define the active contribution sum recursively by

$$F_0(b,a) = \begin{cases} P_a & b = a, \\ 0 & b \neq a, \end{cases}$$
$$F_{n+1}(b,a) = \sum_{c:\, A_{bc}\neq 0} A_{bc} F_n(c,a).$$

The checked identity is

$$\boxed{P_b A^n P_a = F_n(b,a) \qquad (n \geq 0).}$$

Its proof is induction using the exact composition theorem, not an assumed support recurrence. When no active walk exists, the corresponding contribution sum is zero, and hence the actual block of A^n vanishes.

Nilpotence certificate. If no walk of length $h+1$ exists between any pair of packets, then

$$\boxed{A^{h+1} = 0.}$$

Every block vanishes, and reconstruction forces the whole operator to vanish. A second theorem derives the required absence of walks from a supplied level map

$$\ell : I \longrightarrow \{0, \ldots, h\}, \qquad A_{bc} \neq 0 \implies \ell(c) < \ell(b).$$

It proves that a length-n walk increases the level by at least n. This certifies the acyclic case with a bounded height. This block does not implement or verify an algorithm that constructs a topological ordering or computes the longest path.

Polynomial support. For the actual finite sum

$$p(A) = \sum_{j=0}^{d} c_j A^j,$$

the checked support lies in the union of the support relations of the powers. The formalization treats the $j = 0$ contribution explicitly, so the constant term is not accidentally omitted.

Cancellation control. Lean verifies the rational example

$$A = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \qquad A^2 = 0.$$

The coordinate block A_{00} is nonzero, and a length-two active walk from zero to zero exists. Nevertheless the corresponding block of A^2 is zero. This is an exact counterexample to replacing support inclusion by equality.

3. Low-rank algebra and inversion

Source items 5.1, 5.3, and 5.4

Let $A = D_A + U_A V_A^*$ and $B = D_B + U_B V_B^*$ have rectangular factor widths r_A and r_B. The formalization constructs

$$U_C = \begin{bmatrix} D_A U_B + U_A (V_A^* U_B) & U_A \end{bmatrix},$$
$$V_C = \begin{bmatrix} V_B & D_B^* V_A \end{bmatrix}$$

and proves

$$\boxed{AB = D_A D_B + U_C V_C^*.}$$

These are actual concatenated matrices with $r_B + r_A$ columns. Expanding the product and grouping the terms with common right factors proves the identity. The separate rank theorem establishes

$$\operatorname{rank}(AB - D_A D_B) \le r_A + r_B.$$

It bounds actual matrix rank, not just the number of stored columns. Packet diagonality of $D_A D_B$ follows from diagonality of both factors and the checked block-composition formula.

Woodbury. For D and

$$S = I + V^* D^{-1} U$$

both invertible, Lean checks the ordinary matrix inverse formula

$$\boxed{(D + UV^*)^{-1} = D^{-1} - D^{-1} U S^{-1} V^* D^{-1}.}$$

The proof specializes Mathlib's classical Woodbury identity. A separate theorem derives invertibility of $D + UV^*$ from the two stated assumptions. A further theorem verifies that multiplying the proposed inverse by any batch Z solves the updated system. Invertibility of the update is not inserted as an extra premise.

Norm bound. Over the real or complex scalars, all matrix norms in the following theorem are Hilbert-space operator norms, including the rectangular factors:

$$\boxed{\left\|(D + UV^*)^{-1}\right\| \le \left\|D^{-1}\right\| + \left\|D^{-1}U\right\| \left\|S^{-1}\right\| \left\|V^* D^{-1}\right\|.}$$

After the checked inverse identity, the proof uses the triangle inequality and two rectangular operator-norm product bounds.

Not covered here. The arbitrary product-chain rank corollary, optimal small-core recompression, singular-value approximation theorem, chain perturbation bounds, and numerical stability of a particular solve implementation are not marked complete by these results. Algebraic representability is not a GPU execution-time guarantee.

4. Packet perturbation certificates

Source item 8.1 and two uniform consequences

Let X be a normed space over a nontrivially normed field. The formal objects are actual continuous linear maps $X \to X$, with their operator norm. Let $a(e)$ identify the source packet of edge e, and suppose

$$\Delta_e = \Delta_e P_{a(e)}.$$

Set

$$A = \sum_e A_e, \qquad \widehat{A} = \sum_e (A_e + \Delta_e).$$

The exact operator and vector identities are proved first:

$$\widehat{A} - A = \sum_e \Delta_e P_{a(e)}, \qquad (\widehat{A} - A)x = \sum_e \Delta_e (P_{a(e)} x).$$

Applying the finite triangle inequality and the defining operator-norm estimate gives

$$\boxed{\left\|(\widehat{A} - A)x\right\| \le \sum_e \|\Delta_e\| \left\|P_{a(e)} x\right\|.}$$

If $\|\Delta_e\| \le \varepsilon_e \|A_e\|$, then the formal relative certificate is

$$\boxed{\left\|(\widehat{A} - A)x\right\| \le \sum_e \varepsilon_e \|A_e\| \left\|P_{a(e)} x\right\|.}$$

The source additionally assumes that each exact block is source supported. The proof of this bound does not need that assumption. The checked statement therefore uses weaker sufficient hypotheses without weakening the conclusion.

Uniform operator estimates. Two additional checked consequences are

$$\left\|\widehat{A} - A\right\| \le \sum_e \|\Delta_e\| \left\|P_{a(e)}\right\|$$

and, when $\|P_a\| \le 1$ for every packet,

$$\boxed{\left\|\widehat{A} - A\right\| \le \sum_e \|\Delta_e\|.}$$

These estimates are uniform in the input vector and can be used as inputs to later chain analyses. The chain results themselves are not included in this block's completed inventory.

Implementation boundary. The maps Δ_e and their bounds must be supplied or justified for a numerical implementation. This formalization does not establish IEEE rounding behavior, prevent overflow or underflow, model a CUDA kernel, or certify the recorded device speedups. Those are distinct claims and require separate evidence.

5. Statement correction and verification

Boundary audit for item 8.4; evidence for this release

Positive budget is necessary. The source minimizes

$$\sum_e c_e \log \frac{1}{\varepsilon_e}, \qquad \varepsilon_e > 0, \qquad \sum_e s_e \varepsilon_e = \tau,$$

with positive sensitivities and costs, but it does not explicitly impose $\tau > 0$. For a nonempty edge set, Lean proves

$$s_e > 0 \text{ and } \varepsilon_e > 0 \text{ for all } e \quad \Longrightarrow \quad \sum_e s_e \varepsilon_e > 0.$$

It follows that every nonpositive budget is infeasible. A one-edge, unit-sensitivity, zero-budget instance is checked as an exact boundary example.

The amended nonempty-index statement should require $\tau > 0$. Nonemptiness also makes the sum of positive costs in the displayed optimizer's denominator positive. The empty edge set should be treated separately. **Existence, optimality, and uniqueness of the corrected optimizer are not proved in this release.** Item 8.4 remains boundary-audit-only.

Fresh Lean run. All six modules completed with exit code zero under Lean 4.34.0-rc2 and Mathlib revision

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`.

All 60 theorem-level audits and the separate coordinate-projector construction audit use only `propext`, `Classical.choice`, and `Quot.sound`. Warnings were treated as errors. The deliberately false equality and the deliberate admitted-proof placeholder were both rejected. All 17 verifier regression tests passed.

Failures are retained. Nine of the 15 captured development checks failed before correction. Their snapshots and compiler logs remain separate from the active modules and the successful final run. The initial long runtime-extraction command was also interrupted by the host limit. A second extraction completed, and the final run verified the restored runtime and dependency pins.

Reproducibility. The release includes exact sources, module-level commands, source and compiled-artifact hashes, axiom logs, the complete source-result coverage ledger, and a checkpointed verifier. The original CPU suite passed 37,392 checks. It does not reproduce GPU timings.

Remaining scope. Forty-six manuscript-proved items remain not fully covered, plus four separately tracked classical inputs. Mathlib's imported proofs were not independently rechecked, and earlier papers were not rebuilt in full. The next work can build on these semantics to address the cost models, compiler, recompression, and numerical-analysis statements without treating empirical performance as a theorem.